PYDANTIC AI

Long decode. Short Python.

Pydantic AI follows Gemini's continuation token for you.

ONE AGENT CALL — PYTHON

```python
from pydantic_ai import Agent
from pydantic_ai.settings import ModelSettings

settings = ModelSettings(max_tokens=100_000)
agent = Agent(
    'google:gemini-4-argon',
    model_settings=settings,
)
result = agent.run_sync('Write a detailed plan.')
```

GENERATECONTENT · ONE LOGICAL RESPONSE

1 **Generate** — 32K single-request cap · unary — CONTINUATION

↳ carry the continuation token forward

2 **Continue** — Same generation, remaining budget — STOP

✓ Return the combined response

CONTINUATION resumes; MAX_TOKENS stops. Schematic.

pydantic.dev/docs/ai/models/google

GEMINI 4 ARGON

PYDANTIC AI

Hard problems.
More room to reason.

Give Gemini 4 a larger budget for thinking and writing.

256K

EXAMPLE OUTPUT BUDGET
A CEILING, NOT A TARGET LENGTH

GEMINI 4 · GENERATECONTENT — PYTHON

```python
from pydantic_ai import Agent
from pydantic_ai.settings import ModelSettings

settings = ModelSettings(
    thinking='high', max_tokens=256_000
)
agent = Agent('google:gemini-4-argon', model_settings=settings)
result = agent.run_sync('Design a migration plan.')
```

pydantic.dev/docs/ai/models/google

max_tokens covers thinking + output.

GEMINI 4 ARGON

PYDANTIC AI

One window for the whole job.

Long decoding uses the room remaining after input.

1M

token context window
shared by the whole generation

GEMINI 4 ARGON
SHARED CAPACITY

Original input

The first prompt and context

Thinking

The model's reasoning

Output

The answer you receive

`original input + thinking + output ≤ 1,000,000 tokens`

Blocks illustrate categories, not token proportions.

PYDANTIC AI

Stream a longer answer. Keep one Agent call.

Continue past a stream's request limit through Pydantic AI.

● GENERATECONTENT · STREAMING PYTHON

```python
agent = Agent(
    'google:gemini-4-argon',
    model_settings=ModelSettings(
        max_tokens=100_000
    ),
)
async with agent.run_stream('Write a plan.') as run:
    async for chunk in run.stream_text(delta=True):
        print(chunk, end='')
```

ILLUSTRATIVE STREAM · ONE AGENT RUN

Request 1 → Continue → Request 2

ARCHITECTURE PLAN

› Define the boundaries
› Map the failure cases
› Specify the interfaces

64K per stream request · continuation stays in the run.

Request → continuation → combined result

PYDANTIC AI

Long reasoning.
Tools in the loop.

Give a coding agent room to read, reason and plan.

● LONG DECODE + TOOL USE — PYTHON

```python
agent = Agent(
    'google:gemini-4-argon',
    model_settings=settings,
)

@agent.tool_plain
def read_design() -> str:
    return Path('design.md').read_text()

result = agent.run_sync('Plan the migration.')
```

ILLUSTRATIVE CODING-AGENT WORKFLOW

1. **Ask for the design**
   Gemini requests the Python tool.
2. **Read the document**
   Python returns the project context.
3. **Reason through the migration**
   Long decode supplies a larger generation budget.
4. **Return the plan**
   The familiar Agent result.

Imports/settings omitted

pydantic.dev/docs/ai/models/google

GEMINI 4 ARGON